PROGRAMLAMA LABORATUVARI 1

PROJE 1

Farahnozkhon Dilovarovna Masumova
Kocaeli Üniversitesi
220201142

Nazire İpek Agan
Kocaeli Üniversitesi
230201024

Özet

Bu projede JSON türündeki dosyalardan manuel olarak veri ayrıştırması yapılarak, bunları ilgili formüllere göre hesaplamalara dahil edip her turda birimlerin sağlık puanını güncelleyen ve sağlık puanı sıfıra ulaşan tarafın kaybettiği, C programlama dilinde bir oyun simülasyonu yaratıldı. Oyunun başlangıç ve sonundaki birim değerleri iki farklı görsel ile gösterildi.

1.Giriş

Oyun simülasyonunda İnsan İmparatorluğu ve Ork Lejyonu'nun karşı karşıya geldiği ve belirli hesaplamalara göre kazanan tarafın belli olduğu bir kurgu yapıldı. Oyunda kullanılacak verilere ulaşmak için verilen dört adet JSON dosyasıyla beraber ve oyunun taraflarının sayılarını ve özelliklerini belirleyen JSON tipinde bir senaryo dosyası kullanıldı. JSON (JavaScript Object Notation) dosyaları, verileri saklamak için bir veri formatıdır. Dilden bağımsız olarak kullanılabilen ve insanlarca okuması da kolay olduğundan genellikle web uygulamaları, veri tabanı sistemleri ve birçok alanda tercih edilen JSON formatı, anahtar-değer çiftlerinden oluşan nesneler ve sıralı değerler içeren diziler şeklinde yapılandırılmıştır. Bu projede verilen unit_types.json dosyasından savaşa dahil olabilecek birimlerin sayıları ve özellikleri alındı. Örneğin, “insan_imparatorlugu” anahtarının değeri birçok anahtar-değer yapısından oluşur. “piyadeler” anahtarının değeri de aynı şekilde çeşitli anahtar değer çiftleriyken, “saldiri” anahtarının değeri 30'dur. Bu yöntemle İnsan İmparatorluğunun piyadeler biriminin saldırı gücünün 30 olduğu bilgisine ulaşıldı. Benzer şekilde heroes.json dosyasından her bir tarafın kahramanlarına ve etkilerine, creatures.json'dan her bir tarafın canavarlarına ve etkilerine, research.json'dan ise tüm araştırma seviyelerine ve hangi birimlere ne kadar etki ettiğine ulaşıldı. Daha sonra CURL kütüphanesi kullanarak seçilen URL'den indirilen senaryo dosyasına göre oyun başında hangi tarafın ne kadar birime; hangi kahraman, canavar ve araştırma seviyesine sahip olduğu belirlendi. Eldeki bu verilere göre ilgili formüllerle hesaplamalar yapıldı.

2.Yöntem

2.1. cURL ile Senaryo Dosyasının İndirilmesi

HTTP'den dosya indirmek için cURL bünyesinde bulunan libcurl kütüphanesini kullanıldı. cURL; HTTP, HTTPS, FTP, FTPS vb. birçok protokolde veri alımı ve gönderimi için kullanılan açık kaynak bir kütüphanedir. MSYS2 aracılığıyla sonra “pacman -S mingw-w64-x86_64-curl" komutuyla libcurl kütüphanesi indirildi. Bu kütüphane aracılığıyla dosya indirme işlemlerini gerçekleştirmek için dosya_indirme() adlı, void türünde, parametre almayan bir fonksiyon oluşturuldu. Koda #include <curl/curl.h> ile libcurl kütüphanesi dahil edildi. Senaryo olarak kullanılabilecek on dosyanın internet adresleri “linkler” adlı bir dizi içerisinde saklandı. Bu kütüphane aracılığıyla dosya indirebilmek için komut satırında derleme yaparken –lcurl eklendi. Kullanıcıdan linkler dizisinin bir indeksini girmesi istendi. Girilen sayı bir ile on arasındaysa dizinin girilen sayı indeksindeki linkin içeriğindeki JSON dosyası indirilip "senaryo.json" isimli bir dosyaya kaydedildi. Kullanıcının yanlış sayı girme durumu da hesaba katılarak bir ila on arasında bir sayı girilene kadar kullanıcıdan giriş yapması istendi.

2.2. JSON Dosyalarının Ayrıştırılması

Oyun simülasyonunda gerekli olacak verileri hesaplamada kullanabilmek için JSON dosyaları ayrıştırıldı. JSON dosyalarını ayrıştırabilmek için çeşitli fonksiyonlar ve sonra bu ayrıştırılan verileri saklayabilmek için struct yapıları kullanıldı.

Farklı fonksiyonların içinde çağırılan *void adi_temizle(char *str)* fonksiyonu JSON dosyasından alınan string değerlerini temizlemek için kullanıldı. İçerisinde çağırılan *tirnak_temizle()* ile tırnak işaretleri kaldırdı, *void kes_bic(char *str)* ile baştaki ve sondaki boşlukları temizledi ve *parantez_temizle()* ile sondaki süslü parantezleri kaldırdı.

*void json_taraf_ayristirma(const char *s)* fonksiyonu, senayo.json dosyasından okunan verileri ayrıştırarak taraflara yani Ork Legi ya da İnsan İmparatorluğuna ait bilgileri, savaş başlangıç durumunu belirlemek adına bu iki taraf için de araştırma, kahramanlar, canavarlar, birim sayıları gibi bilgileri OyunYapisi tipindeki ilk_struct değişkenine kaydetti.

*void creatues_json_ayristirma(const char *s)* fonksiyonu, creatures.json dosyasından gelen bilgileri ilk_strcut'a kaydetti. İki tarafın da canavarlarının adları, etki türleri, etki değerleri ve açıklaması kaydedilmiş oldu. Ancak canavarların hangi tarafa ait olduğu kaydedilmedi, program bunu senaryo.json'dan gelen verilere göre hesaplamaya dahil etti.

*void heroes_json_ayristirma(const char *s)* fonksiyonu, heroes.json dosyasından gelen bilgileri ilk_struct'a kaydetti. Aynı şekilde iki tarafın da kahramanlarının adları, bonus türleri, bonus değerleri ve açıklamaları kaydedildi. Kahramanların hangi tarafa ait olduğu bilgisi ise yine senaryo.json'dan elde edildi.

*void research_json_ayristir(const char *s)* fonksiyonu, research.json dosyasından gelen bilgileri *ilk_struct*'a kaydetti. Araştırmaların adı, seviyesi (1, 2, 3), türü ve açıklaması saklandı. Senaryodan gelen araştırmaya göre, araştırmanın hangi birim türlerine etki ettiği araştırma açıklamasından sonradan çekildi ve hesaplama ona göre yapıldı.

*void unit_types_json_ayristir(const char *s)* fonksiyonu, unit_types.json dosyasından gelen bilgileri *ilk_struct*'a kaydetti. Birimlerin adı, saldırı, savunma, sağlık, kritik şans değerleri saklandı. Hangi birimin senaryoda mevcut olduğuna ve sayısına göre, unit_types_json'dan alınan verilere göre hesaplamalar yapıldı.

Bu fonksiyonlar sayesinde ana dört JSON dosyası ve senaryo dosyası ayrıştırılmış, veriler daha sonra hesaplama aşamasında kullanılmak üzere *ilk*_strcut yapısına saklanmış oldu. Aynı zamanda görselleştirmenin ilk aşamasında kullanılacak veriler de *ilk_struct* yapısından alındı.

2.3.Hesaplama Mekaniği

Başlangıçta main() fonksiyonunda dosyaların tek tek okunmasıyla veriler *ilk_struct*'a kaydedildi ve daha sonra *savas_ana_fonksiyo*n çağrıldı.

void savas_ana_fonksiyon (OyunYapisi a)* fonksiyonu, tüm savaşın kontrol mekanizması olarak tasarlandı. Savaşın başlatılması, turlarda saldıran tarafın belirlenmesi ve en sonunda kazanan tarafın belirlenmesi bu fonksiyon ile gerçekleştirildi.

İçerisinde savaşın en başında bir kez kullanılan *void savasi_baslat(SavasTarafYapisi *insanlar, SavasTarafYapisi *orklar, OyunYapisi *a)* fonksiyonu, ilk turda insanların saldırı, orkların savunma yapacağı bilindiğinden insan imparatorluğu verilerini *SavasTarafYapisi* tipindeki *saldiran* struct'ına, ork lejyonu verilerini ise *savunan* struct'ına kaydetti. Eldeki ilk veriler *savas_durumunu_yazdir* fonksiyonu ile yazdırıldıktan sonra, etkileri en başta uygulanacak olan kahraman, canavar ve araştırma seviyelerini uygulamak için sırasıyla *arastirma_uygula(saldiran, yapi), arastirma_uygula(savunan, yapi),*

insan_canavar_ve_etkileri(saldiran, yapi) ve *ork_kahraman_ve_canavar_etkileri(savunan, yapi*) fonksiyonları bu parametrelerle çağrıldı.

void arastirma_uygula(SavasTarafYapisi *x, OyunYapisi *yapi) fonksiyonu ile oyun başında senaryodan gelen araştırma uygulandı ve doğrudan ilgili birimin ilgili özelliği güncellendi. Aynı şekilde canavar ve kahraman etkileri de senaryodan gelen veriye göre oyun başında bir kez, doğrudan ilgili birimin ilgili özelliğine etki etti.

Savaş başladıktan sonra güncellenmiş veriler tekrar yazdırıldı ve tarafların toplam sağlıkları hesaplandı. Savaş, tarafların birinin toplam sağlığı sıfıra düşene kadar sürecek şekilde kurgulandı. İlk saldırıyı insanlar yaptığı için, tur sayısı tek olduğunda insanlar, çift olduğunda orklar saldırdı. Saldırı sırası kimdeyse saldıran olarak *savas_tur_uygulamaları* fonksiyonuna gönderildi.

*void savas_tur_uygulamalari(SavasTarafYapisi *saldiran, SavasTarafYapisi *savunan, OyunYapisi *yapi, int tur_sayisi, int saldiri_sayisi)* fonksiyonu her turda gerekli kontrolleri gerçekleştirdi ve toplam savunma, saldırı, net hasar değerlerini hesapladı. Önce 5 turda bir etki etmesi gereken yorgunluk kontrolü *yorgunluk_uygula(SavasTarafYapisi *a, int turSayisi)* fonksiyonu ile yapıldı, sonra *float kritik_uygula(SavasTarafYapisi *saldiran, int turSayisi, int saldiriSayisi)* fonksiyonundan dönen değeri kritik_etkisi adlı bir float değişkene kaydedildi. Bu fonksiyondan kritik vuruş yapılırsa 1.5, yapılmazsa 1.0 değeri döndü. Daha sonra *float net_hasar_hesaplama(SavasTarafYapisi *saldiran, SavasTarafYapisi *savunan)* fonksiyonundan dönen değer ile kritik_etkisi değişkenin çarpımı net_hasar isimli float türünde bir değişkene kaydedildi. Bu değişken ile savunma yapan yani *savunan, void hasari_dagit(SavasTarafYapisi *savunan, float netHasar)* fonksiyonuna gönderildi ve hasar dağılımı gerçekleştirilerek birimin yeni sağlık değeri güncellendi. Güncellenen birim değerleri ve toplam saldırı ve savunma sayısı her tur yazdırıldı.

Böylece kahraman, canavar, araştırma etkileri en başta uygulanıp doğrudan birimlerin özelliklerine etki ederken kritik vuruş uygulaması, yorgunluk her tur kontrol edilip tur sayısı gelince uygulandı. Her tur net hasar, hasar dağılımı ve toplam sağlık hesaplanarak savaş devam ettirildi.

Herhangi bir tarafın toplam sağlık puanı 0'a düşünce kazanan isimli SavasTarafYapisi türünde bir struct'a kazanan tarafın bilgileri kaydedildi ve savaş sonucunu yazdırmak için kullanıldı.

2.4. Görselleştirme

Oyunun başlangıç durumu ve son durumu 10x10 bir ızgara üzerinde görselleştirildi. İnsan ve ork birimleri için ayrı PNG türünde resimler kullanılırken canavarlar için bir, kahramanlar için bir resim seçildi. Görselleştirme işlemini uygulamak için platformlar arası açık kaynak raylib kütüphanesi kullanıldı. 10x10'luk bir ızgara sistemi kullanılarak 800x600 piksellik bir pencere oluşturuldu. Birimlerin yerleşeceği her hücre 60x60 piksel boyutunda ayarlandı. Resim1() fonksiyonu ile savaşın başlangıç durumunda mevcut olan birimler, kahramanlar ve canavarlar gösterildi; resim2() fonksiyonu ile de savaş kazanan tarafın savaş sonunda hayatta kalan birlikleri ve onların güncel sağlık durumuna göre yüzdeleri gösteren sağlık barları çizildi. Arka plan resmi için background.png, insan birimleri için insan1.png - insan4.png, ork birimleri için ork1.png - ork4.png, kahramanlar için ortak super.png, canavarlar için ortak monster.png, boş hücreler için ise empty.png kullanıldı. Savaşın başında insan imparatorluğu birimleri sol tarafa, ork lejyonu birimleri sağ tarafa rastgele bir hücreye yerleştirildi.

Örnek olarak 3.json senaryosunun başlangıç durum ve son durum görselleri:

2.5. Verileri Dosyaya Yazdırma

Savaş boyunca ekrana yazdırılan tüm bilgiler, fprintf() fonksiyonu kullanılarak savas_sim.txt adlı bir belgeye yazıldı. Bu dosya, “w” modunda fopen ile açılarak tüm savaş bilgilerinin kaydedilmesi sağlandı. Böylece savaş boyunca yapılan hesaplamalar ve tur durumu özetleri bu txt belgesinden de okunabilir duruma geldi.

3. Deneysel Sonuçlar

Yine 3.json örnek senaryosuna bakacak olursak, başlangıçta insan imparatorluğu birimlerinin hepsinden birer adet mevcuttu. Ork lejyonunun da tüm birimlerinden birer tane olmak üzere savaşa başladılar. Bir kahraman ya da canavar mevcut olmadığı için etki uygulanmadı ancak insan imparatorluğunun 1 seviyesinde savunma ustalığı araştırması, ork lejyonunun da 1 seviyesinde saldırı geliştirmesi araştırması vardı. Bu verilere uygun araştırma etkisi uygulaması, ilk turda insanların orklara verdiği hasar miktarı ve savaş sonu çıktısı:

```
Arastirma, Kahraman ve Canavar Etkileri Uygulaniyor...

savunma_ustaligi arastirmasi uygulaniyor (Seviye 1, Bonus: 10%)
  piyadeler biriminin savunma gucu %10 artirdi. Yeni deger: 44.00
  okcular biriminin savunma gucu %10 artirdi. Yeni deger: 22.00
  suvariler biriminin savunma gucu %10 artirdi. Yeni deger: 33.00
  kusatma_makineleri biriminin savunma gucu %10 artirdi. Yeni deger: 55.00
saldiri_gelistirmesi arastirmasi uygulaniyor (Seviye 1, Bonus: 10%)
  ork_dovusculeri biriminin saldiri gucu %10 artirdi. Yeni deger: 38.50
  mizrakcilar biriminin saldiri gucu %10 artirdi. Yeni deger: 49.50
  varg_binicileri biriminin saldiri gucu %10 artirdi. Yeni deger: 60.50
  troller biriminin saldiri gucu %10 artirdi. Yeni deger: 77.00
```

```
Toplam Hasar: 100.00
ork_dovusculeri birimi 20.83 hasar aldi:
ork_dovusculeri biriminin yeni saglik degeri: 79.17 (Birim sayisi: 1)
mizrakcilar birimi 16.67 hasar aldi:
mizrakcilar biriminin yeni saglik degeri: 73.33 (Birim sayisi: 1)
varg_binicileri birimi 29.17 hasar aldi:
varg_binicileri biriminin yeni saglik degeri: 100.83 (Birim sayisi: 1)
troller birimi 33.33 hasar aldi:
troller biriminin yeni saglik degeri: 166.67 (Birim sayisi: 1)
```

```
Savas 13 turda bitti.
Insan Imparatorlugu kazandi!
Kalan toplam saglik: 73.19

Kazanan tarafin kalan birimleri:
piyadeler: 1 adet, her birinin saglik durumu 0.00
okcular: 1 adet, her birinin saglik durumu 24.64
suvariler: 1 adet, her birinin saglik durumu 36.96
kusatma_makineleri: 1 adet, her birinin saglik durumu 11.60
```

Bu çıktılarla ayrıştırma fonksiyonlarının verileri doğru ayırıp doğru yapılara kaydettiği, savaş boyunca hesaplamaların doğru yapıldığı teyit edilmiş oldu.

4. Sonuç

Bu oyun simülasyonu projesi, yazılım geliştirme prensiplerini başarıyla uygulayan bir C programıdır. Proje, JSON dosyalarından veri okuma ve bu verileri ayrıştırma, libcurl ve raylib gibi üçüncü parti kütüphanelerini entegre edip etkili bir biçimde kullanma, saklanan verileri görsel olarak sunma gibi birçok adımı gerçekleştirmektedir. İki tarafın etkileşimlerini doğru hesaplayıp yazdırarak savaşı ilerleten bu program, görselleştirme sayesinde hesaplama sonucunu başarıyla sunmakta, kullanıcıya teknik karmaşıkların aksine kolay bir deneyim sunmaktadır. Aynı zamanda veri yapıları, bellek yönetimi, dosya işlemleri gibi birçok ana programlama mekaniğini başarılı bir şekilde uygulayarak, bir öğrenci için programlama bilgilerini gösterebildiği bir ilk proje denemesi olmuştur.

5.Kaynaklar

- *"JSON - Introduction". W3schools. https://www.w3schools.com/js/js_json_intro.asp Erişim tarihi: 24 Ekim 2024.*
- *"libcurl your network transfer diary". Curl. https://curl.se/libcurl/ . Erişim Tarihi: 24 Ekim 2024.*
- *"cURL Nedir Nasıl Kullanılır?". Ceyhun Enki Aksan. 30 Nisan 2020. https://ceaksan.com/tr/curl-nedir-nasil-kullanilir .*
- *"raylib is a simple and easy-to-use library to enjoy videogames programming". raylib. https://www.raylib.com/ . Erişim Tarihi: 24 Ekim 2024.*
- *Yorulmaz, Muhhammet. Programlamayı C ile Öğreniyorum: Güncellenmiş 8. Baskı. Ankara: Palme Yayınevi, 2022. s. 394- 411.*
- *Yorulmaz, Muhhammet. Programlamayı C ile Öğreniyorum: Güncellenmiş 8. Baskı. Ankara: Palme Yayınevi, 2022. s. 358 - 367.*
- *"MSYS2 Packages". MSYS2. https://packages.msys2.org/packages/mingw-w64-x86_64-curl Erişim Tarihi: 24 Ekim 2024.*
- *"Parse JSON message manually in C". Stackoverflow. https://stackoverflow.com/questions/41570407/parse-json-message-manually-in-c .*
- *"Basic JSON Deserializer in C.". Dinesh Vujuru. 25 Temmuz 2016. https://www.youtube.com/watch?v=04LjWSeMCQM&list=WL&index=49&t=17s*
- *"JSON Serializer Code Execution in C". Mahikanth Nag Yalamarthi. 1 Ağustos 2016. https://www.youtube.com/watch?v=XDUOxp73PdM*
- *"Getting started with libcurl and curlpp for http requests in C and C++". Facile Tutorials. 6 Temmuz 2024. https://www.youtube.com/watch?v=a1LjsucKZwg&list=WL&index=52*
- *"How to compile Raylib from scratch and set up your first projects (Windows / C) in 15 minutes". Rudy Faille. 27 Ocak 2021. https://www.youtube.com/watch?v=HPDLTQ4J_zQ&list=PPSV*
- *insan1.png, https://www.pngegg.com/en/png-cfmqu*
- *insan2.png, https://www.pngegg.com/en/png-yaagj*
- *insan3.png, https://www.cleanpng.com/png-the-witcher-3-wild-hunt-geralt-of-rivia-playstatio-906382/*
- *insan4.png, https://www.pngegg.com/en/png-tmmos*
- *ork1.png, https://www.pngwing.com/en/free-png-kkmos*
- *ork2.png, https://www.pngwing.com/en/free-png-vcqnt*
- *ork3.png, https://www.pngwing.com/en/free-png-vftft*
- *ork4.png, https://www.pngwing.com/tr/free-png-tlhnq*
- *super.png, https://www.pngegg.com/en/png-hpsvi*
- *monster.png, https://www.pngwing.com/ru/free-png-afvpf*
- *background.png, https://gamefound.com/en/projects/battle-systems/maladum-dungeons-of-enveron/products/details/37181*
- *empty.png https://www.freepik.com/premium-vector/white-grey-checkered-seamless-pattern_21804310.htm*